Python Niveau I

Python Niveau I — Syllabus

Initiation au langage Python

Durée : 2 jours (14 heures) **Format** : Présentiel ou distanciel synchrone

Description

Formation de deux jours pour découvrir Python et son écosystème. Les stagiaires acquièrent les bases pour écrire des scripts, manipuler les structures de données fondamentales, lire et écrire des fichiers, structurer du code maintenable, et produire des visualisations simples avec matplotlib.

Public cible

- Analystes de données débutants en Python
- Professionnels migrant depuis Excel, SPSS ou SAS
- Étudiants en data science, biologie, économie, sciences sociales
- Professionnels souhaitant automatiser des traitements de données

Prérequis

- Notions de statistique descriptive (moyenne, médiane, écart-type)
- Aisance informatique de base
- Aucune expérience en programmation requise

Objectifs pédagogiques

1. **Installer et configurer** Python et un environnement de développement (VS Code, Jupyter)
2. **Utiliser l'interpréteur interactif** et écrire des scripts .py
3. **Identifier et manipuler** les types de données : int, float, str, bool, None
4. **Maîtriser les structures** : listes, tuples, dictionnaires, ensembles
5. **Écrire des fonctions** et utiliser les structures de contrôle (if, for, while)
6. **Lire et écrire des fichiers** (texte, CSV, JSON)
7. **Importer et utiliser des modules** standards et externes (pip)
8. **Structurer un mini-projet Python** : fonctions, modules, validations et gestion d'erreurs
9. **Lire et transformer des données** en Python pur (csv, dict, list, agrégations)
10. **Créer des visualisations** avec matplotlib : line, histogramme, barres, scatter
11. **Livrer un mini-projet de synthèse** : lecture CSV, indicateurs, export JSON, visualisation

---

Programme

Jour 1 — Découverte de Python

Module	Contenu
1. Introduction à Python	Installation, environnements (VS Code, Jupyter), interpréteur, scripts, pip, aide
2. Types et structures de données	int, float, str, bool, listes, tuples, dictionnaires, ensembles, indexation, slicing
3. Fonctions et contrôle	if/elif/else, for, while, range, fonctions def, arguments, compréhensions de liste
4. Atelier pratique	Exercices guidés consolidant les modules 1 à 3

Jour 2 — Manipulation de données

Module	Contenu
5. Fichiers et modules	open/read/write, with, csv, json, os.path, import, modules standards
6. Structuration du code	Organisation en fonctions/modules, validation des entrées, gestion d'erreurs, configuration simple
7. Visualisation avec matplotlib	scatter, hist, boxplot, bar, subplots, personnalisation, export
8. Projet de synthèse Python pur	Flux complet csv -> nettoyage -> indicateurs -> export JSON -> visualisation

---

Compétences acquises

Compétence	Niveau
Environnement Python et scripts	Maîtrise
Types et structures de données	Maîtrise
Fonctions et contrôle de flux	Maîtrise
Fichiers et modules	Autonomie
Structuration et modularité du code	Autonomie
Traitement de données en Python pur	Autonomie
Visualisation (matplotlib)	Initiation

Évaluation

- Exercices pratiques à la fin de chaque module
- Projet intégrateur en fin de formation (Module 8)
- Attestation de participation

Méthodes pédagogiques

- Vidéos animées par module avec narration et code en direct
- Démonstrations live dans VS Code ou Jupyter
- Exercices progressifs (facile → intermédiaire → avancé)
- Projet intégrateur sur données réelles
- Slides, scripts Python commentés, fiches mémo

Définitions essentielles

- **Variable** : nom qui référence une valeur en mémoire.
- **Type** : nature d'une valeur (`int`, `float`, `str`, etc.).
- **Fonction** : bloc de code réutilisable défini avec `def`.
- **Liste** : collection ordonnée et mutable d'éléments.
- **Dictionnaire** : collection clé/valeur pour un accès direct.

Notions théoriques supplémentaires

- **Mutabilité** : différence entre objets modifiables (list, dict) et immuables (str, tuple).
- **Référence vs copie** : impact des affectations et des copies superficielles/profondes.
- **Portée des variables** : local, global, fermeture lexicale.
- **Complexité de base** : coût moyen des opérations courantes sur listes et dictionnaires.
- **Exceptions** : modèle `try/except/finally` et importance des messages explicites.

Glossaire technique détaillé (niveau I)

- **Interpréteur** : programme qui lit et exécute le code Python instruction par instruction.
- **Script** : fichier `.py` exécutable contenant une suite d'instructions.
- **Module** : fichier Python importable depuis un autre script.

- **Package** : dossier de modules organisés par domaine fonctionnel.
- **Signature de fonction** : nom + paramètres + valeur de retour attendue.
- **Type hint** : annotation facilitant la lecture du contrat de fonction.
- **Valeur sentinelle** : valeur spéciale (ex: `None`) indiquant un état particulier.
- **Idempotence** : exécuter plusieurs fois une même opération sans changer indéfiniment le résultat.
- **Validation** : contrôle systématique des entrées avant traitement métier.
- **Traceback** : pile d'appel affichée lors d'une exception.

Détails techniques complémentaires

- Préférer des fonctions courtes (5–20 lignes) avec un objectif unique.
- Nommer explicitement les variables métier (`montant_ttc`, `date_commande`).
- Éviter les effets de bord: copier une liste avant modification si l'original doit rester intact.
- Distinguer erreurs de format (entrée invalide) et erreurs d'exécution (fichier absent, permission).
- Centraliser les constantes et chemins en haut de script pour faciliter maintenance et tests.
- Documenter les hypothèses (unité, format de date, valeurs obligatoires) au plus près du code.

Exemples de code (niveau 1)

Condition + boucle

```
notes = [8, 12, 15, 9]
for n in notes:
    if n >= 10:
        print(f"{n} valide")
    else:
        print(f"{n} à renforcer")
```

Fonction simple

```
def moyenne(valeurs):
    return sum(valeurs) / len(valeurs)

print(moyenne([10, 12, 14]))
```

Lecture CSV en Python pur

```
import csv

with open("ventes.csv", "r", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
print(rows[:2])
```

Gestion d'exception simple

```
def to_int(value: str) -> int:
    try:
        return int(value)
    except ValueError as err:
        raise ValueError(f"valeur invalide: {value}") from err
```

Outils

Outil	Version
Python	≥ 3.11
VS Code ou Jupyter Notebook	Dernière version
pip	Inclus avec Python
matplotlib	≥ 3.7
random/statistics (bibliothèque standard)	Inclus avec Python
csv/json (bibliothèque standard)	Inclus avec Python